```
(*
                            CS51 Lab 14
              Lazy Programming and Infinite Data Structures
                    Implementing laziness as user code
 *)

(*
                                SOLUTION
 *)

(*======================================================================
Readings:

    This lab builds on material from Chapter 17 through Section 17.2
    of the textbook <http://book.cs51.io>, which should be read before
    the lab session.

======================================================================*)

(* This lab provides practice with delayed (lazy) computations,
implemented as user code using the naturally lazy behavior of OCaml
functions. (In the next lab we explore OCaml's built-in `Lazy`
module.)

In this lab, you will use an infinite data structure, the *stream*. *)

open CS51Utils ;; (* for access to timing functions *)

  (* Digression: We've just opened `CS51Utils` above, so we can make
     use of its components (like `Absbook.call_reporting_time`) in
     this file. But if you want to use it in the REPL, you'll need to
     make it accessible there as well, e.g.,

        # #require "CS51Utils" ;;
        # open CS51Utils ;;                                        *)

(*======================================================================
Part 1: Programming with lazy streams

Recall the lazy `stream` type and associated functions from the
reading, here packaged up into a module. *)

(* An aside: The solutions here were chosen for simplicity, not for
   efficiency. For instance, the definitions of `first`, `smap`, and
   `smap2` all force their stream argument (that is, apply it to `()`)
   more than once. (The forces occur implicitly in the calls to `head`
   and `tail`.) A more efficient implementation would force only once,
   saving the results. For instance,

    let rec smap (f : 'a -> 'b) (s : 'a stream) : ('b stream) =
      fun () ->
        let Cons (h, t) = s () in
```

```
        Cons (f h, smap f t) ;;

   Of course, in an implementation that uses memoizing thunks instead
   of unit functions to delay computation, this problem of redundancy
   of computation is eliminated. The first force of `s` (whichever
   call it arises from) causes the result to be cached and thereby
   made immediately available for the second force. For this reason,
   we'll mostly ignore this issue of multiple forces in the solutions
   we give in this lab, though we comment on it with alternative
   solutions below. *)

module LazyStream =
  struct

    type 'a stream_internal = Cons of 'a * 'a stream
     and 'a stream = unit -> 'a stream_internal ;;

    (* head strm -- Returns the first element of `strm`. *)
    let head (s : 'a stream) : 'a =
      let Cons (h, _t) = s () in h ;;

    (* tail strm -- Returns a stream containing the remaining elements
       of `strm`. *)
    let tail (s : 'a stream) : 'a stream =
      let Cons (_h, t) = s () in t ;;

    (* first n strm -- Returns a list containing the first `n`
       elements of the `strm`. *)
    let rec first (n : int) (s : 'a stream) : 'a list =
      if n = 0 then []
      else head s :: first (n - 1) (tail s) ;;

    (* smap fn strm -- Returns a stream that applies the `fn` to each
       element of `strm`. *)
    let rec smap (f : 'a -> 'b) (s : 'a stream) : ('b stream) =
      fun () -> Cons (f (head s), smap f (tail s)) ;;

    (* smap2 fn strm1 strm2 -- Returns a stream that applies the `fn`
       to corresponding elements of `strm1` and `strm2`. *)
    let rec smap2 (f : 'a1 -> 'a2 -> 'b)
                  (s1 : 'a1 stream)
                  (s2 : 'a2 stream)
                : 'b stream =
      fun () -> Cons (f (head s1) (head s2),
                      smap2 f (tail s1) (tail s2)) ;;
  end ;;

(* We open the module for ease of access throughout this lab. *)

open LazyStream ;;

(* Here, recalled from the reading, is the definition of an infinite
stream of ones. *)

```

```
let rec ones : int stream =
  fun () -> Cons (1, ones) ;;

(* Now you'll define some useful streams. Some of these were defined
in the reading, but see if you can come up with the definitions
without looking them up. *)

(*.....................................................................
Exercise 1: An infinite stream of the integer 2. As usual, for this
and all succeeding exercises, you shouldn't feel beholden to how the
definition is introduced in the skeleton code below. (We'll stop
mentioning this now, and forevermore.)
.....................................................................*)

(* We can implement `twos` exactly parallel to how `ones`
   was implemented above:

      let rec twos : int stream =
        fun () -> Cons (2, twos) ;;

   But a more elegant definition makes use of the stream manipulation
   function `smap`. *)

let twos : int stream =
  smap succ ones ;;

(* Note the addition of the `rec` keyword in the first definition,
   not needed in the second. *)

(*.....................................................................
Exercise 2: An infinite stream of threes, built by summing the ones
and twos.
.....................................................................*)

let threes : int stream =
  smap2 (+) ones twos ;;

(*.....................................................................
Exercise 3: An infinite stream of natural numbers (0, 1, 2, 3, ...).
Try working this out on your own before checking out the solution in
the textbook.
.....................................................................*)

(* Here is the implementation from the textbook, which makes `nats`
   directly as a recursive *value*. *)

let rec nats : int stream =
  fun () -> Cons (0, smap succ nats) ;;

(* An alternative implementation defines a recursive *function*
`nats_from` that generates the natural numbers starting from an
initial value.

    let rec nats_from (start : int) : int stream =
```

```
      fun () -> Cons (start, nats_from (start + 1)) ;;

    let nats : int stream = nats_from 0 ;;

or, hiding the `nats_from` inside the definition of `nats`,

    let nats : int stream =
      let rec nats_from (start : int) : int stream =
        fun () -> Cons (start, nats_from (start + 1)) in
      nats_from 0 ;;

This approach is faster but less "infinite data structure"-y. *)

(*.....................................................................
Exercise 4: Create a function `alternate`, which takes two streams and
'alternates' them together; `alternate` should output a single stream
created by alternating the elements of the two input streams starting
with an element of the first stream.

For example, 'alternating' infinite streams of ones (1,1,1,1....) and
twos (2,2,2,2....) would look like this:

     # first 10 (alternate ones twos) ;;
     - : int list = [1; 2; 1; 2; 1; 2; 1; 2; 1; 2]

and alternating the natural numbers (0,1,2,3,4,...) and ones would
look like this:

     # first 10 (alternate nats ones) ;;
     - : int list = [0; 1; 1; 1; 2; 1; 3; 1; 4; 1]
.....................................................................*)

let rec alternate (s1 : 'a stream) (s2 : 'a stream) : 'a stream =
  fun () -> Cons (head s1, alternate s2 (tail s1)) ;;

(* Now some new examples. For these, you should build them from
previous streams (`ones`, `twos`, `threes`, `nats`) by making use of
the stream mapping functions (`smap`, `smap2`). *)

(*.....................................................................
Exercise 5: Generate two infinite streams, one of the even natural
numbers, and one of the odds.
.....................................................................*)

(* There are several ways of generating a stream of evens. One is to
   implement the cyclic structure directly, as was done with nats
   above:

     let rec evens = fun () -> Cons (0, smap ((+) 2) evens) ;;

   But we asked for building them from existing streams, for instance,
   by mapping a doubling function over the nats:

     let evens = smap (( * ) 2) nats ;;
```

```
   or adding `nats` to itself:
 *)

let evens = smap2 (+) nats nats ;;

(* The odds stream has similar possibilities. Here we just add one to
   all the evens. *)

let odds = smap succ evens ;;

(* In addition to mapping over streams, we should be able to use all
the other higher-order list functions you've grown to know and love,
like folding and filtering. So let's implement some. *)

(*.....................................................................
Exercise 6: Define a function `sfilter` that takes a predicate (that
is, a function returning a `bool`) and a stream, and returns the
stream that contains all the elements in the argument stream that
satisfy the predicate. Here's an example -- generating a stream of
even numbers by filtering the natural numbers for the evens:

   # let evens = sfilter (fun x -> x mod 2 = 0) nats ;;
   val evens : int stream = <fun>
   # first 10 evens ;;
   - : int list = [0; 2; 4; 6; 8; 10; 12; 14; 16; 18]
.....................................................................*)

(* The most straightforward way to implement filtering (though not
   very efficient) is as follows:

    let rec sfilter (pred : 'a -> bool) (s : 'a stream) : 'a stream =
      fun () ->
        if pred (head s) then Cons ((head s), sfilter pred (tail s))
        else (sfilter pred (tail s)) () ;;

   There are multiple alternatives. Perhaps you implemented one of
   these.

   The definition above forces evaluation of `s` (that is, applies `s`
   to `()`) several times (on every call of `head` and `tail`). This
   is the same issue mentioned above in the discussion of `smap`. To
   remove this expensive recomputation, we can do the force once and
   store the result, using the parts as needed:

    let rec sfilter (pred : 'a -> bool) (s : 'a stream) : 'a stream =
      fun () ->
        let Cons (hd, tl) = s () in
        if pred hd then Cons (hd, sfilter pred tl)
        else (sfilter pred tl) () ;;

   This is the version we use below.

   Another set of variations involves moving the test of the head
```

```
   outside the "thunk". In that case, `sfilter` verifies the `pred`
   condition on the head of the stream *before* doing any postponement
   of the tail. It thus eagerly filters out non-pred elements,
   postponing only at the first `pred`-satisfying element. We get:

    let rec sfilter (pred : 'a -> bool) (s : 'a stream) : 'a stream =
      if pred (head s)
      then fun () -> Cons (head s, sfilter pred (tail s))
      else (sfilter pred (tail s)) ;;

   This version can run *much* slower than the one above. Indeed, on a
   stream none of whose elements satisfy `pred`, this `sfilter` will
   never terminate, because the call to `sfilter` in the `else` clause
   isn't postponed at all! That problem can be remedied by delaying
   the else branch as well:

    let rec sfilter (pred : 'a -> bool) (s : 'a stream) : 'a stream =
      if pred (head s)
      then fun () -> Cons (head s, sfilter pred (tail s))
      else fun () -> (sfilter pred (tail s)) () ;;

   Again, these implementations implicitly re-force `s` multiple times
   by virtue of the multiple calls to `head` and `tail`. Instead, we
   can force `s` explicitly once and reuse the results.

    let rec sfilter pred s =
      let Cons (hd, tl) = s () in
      if pred hd
      then fun () -> Cons (hd, sfilter pred tl)
      else fun () -> (sfilter pred tl) () ;;
 *)

let rec sfilter (pred : 'a -> bool) (s : 'a stream) : 'a stream =
  fun () ->
    let Cons (hd, tl) = s () in
    if pred hd then Cons (hd, sfilter pred tl)
    else (sfilter pred tl) () ;;

(*.....................................................................
Exercise 7: Now redefine `evens` and `odds` (as `evens2` and `odds2`)
by using `sfilter` to filter over `nats`.
.....................................................................*)

let even x = (x mod 2) = 0 ;;
let odd x = not (even x) ;;

let evens2 = sfilter even nats ;;
let odds2 = sfilter odd nats ;;

(*======================================================================
Part 2: Eratosthenes' Sieve

Eratosthenes' sieve is a method for generating the prime numbers.
Given a list (or stream) of natural numbers starting with 2, we filter
```

out those in the tail of the list divisible by the head of the
list and then apply the sieve to that tail. The first few steps go
something like this: We start with the natural numbers (in the example
here, just a prefix of them).

```
    2 3 4 5 6 7 8 9 10 11 12 13 14 15...
```

The first element, 2, is prime. Now we remove numbers divisible by 2
from the tail of the list (marking here with a | the boundary between
the first element and the tail we're currently working on:

```
    2  |  3 5 7 9 11 13 15...
```

and apply the sieve to the tail:

```
    2 3  |  5 7 11 13
```

and again:

```
    2 3 5  |  7 11 13
    2 3 5 7  |  11 13
    ...
    2 3 5 7 11 13
```

Here's the process of sieving a stream of numbers in more detail:

1. Retrieve the head and tail of the stream. The head is the first
   prime in the result stream; the tail is the list of remaining
   elements that have not been sieved yet. For instance,

   ```
   head      | tail
   2         | 3 4 5 6 7 8 9 10 11 ...
   ```

2. Filter out all multiples of the head from the tail.

   ```
   head      | filtered tail
   2         | 3 5 7 9 11 ...
   ```

3. Sieve the filtered tail to generate all primes starting with the
   first element of the tail.

   ```
   head      | sieved filtered tail
   2         | 3 5 7 11 ...
   ```

4. Add the head on the front of the sieved results.

   ```
   2 3 5 7 11 ...
   ```

5. Of course, this whole series of computations (1 through 4)
   should be delayed, and only executed when forced to do so.

.....
Exercise 8: Implement Eratosthenes sieve to generate an infinite
stream of primes. Example:

```
# primes = sieve (tail (tail nats)) ;;
# first 4 primes ;;
- : int list = [2; 3; 5; 7]
```

(You probably won't want to generate more than the first few primes this way; it'll take too long, depending on how your other stream functions were implemented. Here are some timings from the solution code on my laptop:

```
  n      time for nth prime (seconds)
  1 --   0.00000405
  2 --   0.00001597
  3 --   0.00006604
  4 --   0.00105000
  5 --   0.00343299
  6 --   0.04916501
  7 --   0.19323015
  8 --   3.12322998
```

Just generating the first eight primes takes over three seconds -- longer if a less efficient `sfilter` is used. You'll address this performance problem in the next lab.)

In defining the `sieve` function, the following function may be useful: *)

```
(* not_div_by n m -- Predicate returns true if `m` is not evenly
   divisible by `n` *)
let not_div_by (n : int) (m : int) : bool =
  m mod n <> 0 ;;
(*............................................................*)

(* A direct implementation of the 5-step process above (with numbers
   keyed to the description) would be:

    let rec sieve s =
      fun () -> Cons (head s, sieve (sfilter (not_div_by (head s)) (tail s))) ;;
        ^         ^     ^        ^      ^                                 ^
        |         |     |        |      |                                 |
        5         4     1        3      2                                 1

   But as in `sfilter` above this forces `s` multiple times. Instead,
   we can force once and save the results:

    let rec sieve s =
      fun () ->
        let Cons (hd, tl) = s () in
        Cons (hd, sieve (sfilter (not_div_by hd) tl)) ;;

   Finally, the force of `s` can be done before delaying the rest of the
   computation, as we do below, or after, as above. (Either way requires
   essentially the same amount of time.) *)
```

```
let rec sieve (s : int stream) : int stream =
  let Cons (hd, tl) = s () in
  fun () -> Cons (hd, sieve (sfilter (not_div_by hd) tl)) ;;

(* With the sieve function in hand, we can generate an infinite stream
   of primes. *)

let primes : int stream = sieve (tail (tail nats)) ;;

(* For testing purposes (and just for fun), we generate a table of
some times to generate primes, stopping as soon as the next prime
takes more than half a second: *)

let prime_timing () =
  print_endline "  n      time for nth prime (seconds)";
  let n = ref 1 in
  let finished = ref false in
  while not !finished && !n < 100 do
    let _l, msecs = Absbook.call_timed (first !n) primes in
    Printf.printf "%3d -- %12.8f\n" !n (msecs /. 1000.);
    n := succ !n;
    if msecs > 500. then finished := true
  done ;;

let _ = prime_timing () ;;
```